Santosh Sai Gowtham Pasala

San Jose, CA | santoshp12122@gmail.com | +1 (669) 209-0322
linkedin.com/in/santoshsaigowtham | github.com/pssgowtham | pssgowthamportfolio.vercel.app

PROFESSIONAL SUMMARY

Software engineer with 4+ years building production systems end to end. Backend focus on JavaScript and **Node.js** microservices, **REST APIs**, and **PostgreSQL**, with full-stack delivery in **React** and Next.js. Build AI backends in **Python** and **FastAPI** on top of that foundation, including **LangGraph** multi-agent orchestration, **RAG** pipelines on **Pinecone**, and **MCP** integrations, backed by LLM evals and cost and latency telemetry. Work directly with stakeholders from problem definition through deployment on Azure and AWS.

TECHNICAL SKILLS

Programming Languages: JavaScript, TypeScript, Python, Java, SQL, C++, HTML5, CSS3, Shell Script.
Backend: Node.js, Express.js, Flask, FastAPI, Pydantic, Spring Boot, Microservices, REST APIs, API Design, gRPC, System Design, Event-Driven Architecture, OAuth2.0/JWT, Unit Testing (PyTest, Jest), Hibernate, JPA.
Frontend: ReactJS, Vue.js, AngularJS, HTML5, CSS3, Bootstrap, Next.js.
Databases: PostgreSQL, MySQL, MongoDB, Redis, DynamoDB, Vector Databases (Pinecone, Weaviate), NoSQL.
Cloud and Infrastructure: AWS (EC2, S3, Lambda, SQS, SNS, CloudWatch, DynamoDB), Azure (Azure OpenAI, Azure AI Foundry, Azure Document Intelligence, Azure AI Search, AKS, Azure Monitor), Docker, Kubernetes, Jenkins, GitHub Actions, CI/CD, Git, Agile.
AI and LLM Systems: LangGraph, LangChain, LangSmith, OpenAI, Claude API, RAG Pipelines, Multi-Agent Orchestration, MCP (Model Context Protocol), Prompt Engineering, Vector Databases (Pinecone, PGVector, Weaviate), Structured Outputs, Model Evaluation and Evals Framework.
Monitoring and Observability: LangSmith, Prometheus, Grafana, Datadog, CloudWatch, Distributed Tracing.

EXPERIENCE

Software Engineer | Geico | San Jose, CA Jul 2024 - Present

- Built backend services with **Python**, **Node.js**, and **FastAPI** that handle 1K+ customer service emails a day, with **REST APIs** for email intake, status checks, and review queues used by the **React** dashboards and internal tools.
- Defined the API design for the backend services, covering URL structure, versioning, error responses, and request validation with Pydantic, so endpoints stayed consistent and easy for other teams to use.
- Designed **PostgreSQL** schemas and stored procedures for conversation state management under high concurrency.
- Delivered a production system end to end, with **Python**/**FastAPI** backend services, **React** dashboards, and a **LangGraph** multi-agent core on **Azure OpenAI** processing 1K+ customer service emails daily at 92% accuracy.
- Added **Kafka** with worker nodes to process customer emails in the background, so spikes in email volume do not slow down the main services.
- Worked with customer-service stakeholders to build the **React** and **Material-UI** dashboards for agent performance monitoring, low-confidence review queues, and real-time approval workflows, cutting triage turnaround time by 40%.
- Set up CI/CD with **Azure Pipelines**, with automated testing, containerized builds, and production deploys at 85%+ test coverage with PyTest and Jest.
- Implemented end-to-end observability across **LangSmith**, **Grafana**, and Azure Monitor for pipeline visibility, debugging, and incident response.
- Developed a production **RAG** pipeline with **LangChain**, **LangGraph**, and **Pinecone**, grounding customer email responses in enterprise knowledge bases and cutting average response time from 4 hours to 12 minutes.
- Used and customized **MCP** servers to connect agents with enterprise knowledge bases and internal APIs, so agents can look up customer data, check campaign history, and run operations.
- Set up a full LLM evaluation framework with **LangSmith**, running automated regression evals over a 200+ case golden dataset and catching 15+ production edge cases before customer impact.
- Evaluated models and routed classification to a smaller, faster model and generation to a larger model for quality, batching low-volume workloads, and tracking per-run token spend.
- Refactored the backend codebase to remove duplication and split large services into smaller reusable modules, making the code easier to test, maintain, and extend.
- Cut boilerplate by 40% by working AI-native with Claude Code and Cursor for code generation, test writing, and refactoring, with every change reviewed and tested before shipping.

Software Analyst | Deloitte | Hyderabad, India Aug 2021 - Aug 2022

- Designed scalable backend infrastructure for enterprise clients supporting 50+ microservices handling millions of daily transactions, with a **Node.js** orchestration layer using request batching, retry logic, and circuit breakers for resilience under high load.
- Created a **Node.js** API gateway and **Python FastAPI** services with OAuth2.0/JWT authentication, rate limiting, and structured logging for secure communication across distributed services, reducing API latency by 25%.
- Optimized database performance across **PostgreSQL**, MongoDB, and Redis, implementing caching strategies, query optimization, and schema design, reducing database load by 30% and improving throughput by 40%.
- Wrote the Swagger/OpenAPI 3.0 contracts for scalable **REST APIs** that 50+ microservices built against, enabling contract-driven development across distributed teams.
- Built Vue.js and ReactJS real-time monitoring dashboards with Redux state management for service health visualization and production troubleshooting, reducing incident response time by 30-35%.
- Worked on Deloitte's SaaS AI Framework, using Datadog to monitor 50+ services, evaluate AI ethics, protect customer privacy, and reduce AI bias by 30%, while communicating insights to client stakeholders and cross-functional teams.
- Analyzed client Model Risk Management projects in healthcare, evaluating model performance, mitigating risks, and upgrading algorithm efficiency by 25%.

Software Engineer | Virtutech | Hyderabad, India May 2020 - Jul 2021

- Developed the core backend services in **Node.js** and **Python** (**FastAPI**) with customer-facing RESTful APIs and third-party integrations for real-time data exchange, improving data retrieval efficiency by 50%.
- Implemented asynchronous event-driven workflows using message queues and background job processing, enabling reliable handling of high-volume transactions and notifications.
- Containerized applications with Docker and deployed services on AWS infrastructure, ensuring consistency across development, staging, and production environments.
- Automated CI/CD with AWS CodePipeline, CodeBuild, and Docker, cutting deployment time by 40% and sustaining 99.9% uptime.
- Secured login and session flows with JWT and OAuth 2.0, tightening application security without breaking the user experience.
- Optimized SQL and NoSQL database queries, indexing strategies, and caching mechanisms, significantly improving API response times and scalability under peak traffic.
- Shipped reusable UI components and cut frontend render times with lazy loading, code splitting, and memoization, reducing page load times by 30%.

PROJECTS

CodeReviewer AI: Autonomous LLM-Powered Pull Request Review Agent

https://github.com/pssgowtham/codereviewer

- Built an autonomous code review agent with tool-calling and **MCP** integration, analyzing 500+ PRs/month and delivering context-aware feedback on security, performance, and maintainability, reducing review time by 60%.
- Integrated Tree-sitter AST parsing with **RAG** (**Pinecone**) for repo-aware insights and deployed via AWS Lambda with DynamoDB and GitHub webhooks, achieving 88% precision on issue detection.

Multi-Agent Research System: AI-Powered Deep Research Platform

https://github.com/pssgowtham/multi-agent-research

- Created a multi-agent research platform using **LangGraph**, with 5 specialized agents and a conditional feedback loop for iterative refinement. Integrated semantic search and **Pinecone** vector memory with similarity-based caching.
- Added a **FastAPI** backend with SSE streaming for real-time agent progress and multi-layered safety guards (LlamaGuard prompt injection detection, PII filtering), monitored with **LangSmith** for end-to-end LLM observability.

EDUCATION

Master of Science in Software Engineering | San Jose State University Aug 2022 - May 2024